SpecField: Checking Field-Level Conformance of Protocol Implementations to Standards

***Abstract*—Field requirements in protocol standards are usually described in natural language, while the corresponding implementation logic is scattered across parsing, validation, and error-handling code, making it difficult to directly connect standard requirements with implementation behavior. This paper presents SpecField, a framework for field-level protocol conformance checking. The core idea is to use protocol fields as semantic anchors between natural-language standards and source-code implementations: SpecField extracts structured field rules from standards, aligns them with variables and code contexts in implementations, and uses traceable evidence to judge whether implementation behavior deviates from standard requirements. For suspected deviations, SpecField further generates maintainer-oriented defect reports and reproducible validation artifacts to support manual confirmation and result auditing. We submitted 204 reports on real protocol implementations including TLS/DTLS and QUIC; 126 have been confirmed or fixed, 9 were rejected as false positives, and 69 remain pending. Two confirmed issues were assigned previously unknown CVEs.**

I. Introduction

Network protocol standards are a key foundation for interoperability and security across independent implementations. Protocols such as TLS [1], [2], MQTT [3], [4], and QUIC [5]–[7] specify not only how endpoints parse messages, but also how they negotiate parameters, update state, authenticate peers, and handle exceptional inputs. A deviation from these requirements does not necessarily become an exploitable vulnerability immediately, but it can still introduce practical risk. For example, an implementation may accept a field value forbidden by the standard, omit a state-dependent check, incorrectly reject a valid message, or trigger abnormal error-handling behavior under a specific configuration. Such deviations may cause interoperability failures and may further enable downgrade attacks, denial-of-service conditions, or other security risks.

Detecting these deviations is difficult because standards and code usually express the same behavior in different forms. Protocol standards are natural-language documents whose requirements are distributed across tables, field descriptions, state-machine descriptions, compatibility notes, and exception clauses. RFC-style keywords can indicate requirement strength [8], [9], and prior work has used RFC-derived rules, parser specifications, and large-language-model agents to analyze protocol implementations [10]–[13]. However, these approaches usually analyze rules, interfaces, or protocol behaviors as their main objects, and their granularity is relatively coarse. They therefore struggle to precisely capture the correspondence between individual fields, their related conditions, and the implementation behavior that realizes them.

This paper focuses on field-level protocol requirements. Many actionable requirements specify that a field may appear only in a particular state, that its value must fall within an allowed range, that it must match another field, that it depends on negotiated parameters, or that it must trigger rejection under an invalid combination. We represent such requirements as $\langle C, f, A\rangle$: under condition C, field f must satisfy action or constraint A. This form is not a complete formal semantics, but it provides a clear boundary for checking standard text against implementation code.

We present SpecField, a conformance-checking framework for natural-language protocol standards and source-code implementations. SpecField first extracts the above rules, then retrieves syntax-aware code contexts related to field parsing, value validation, state dependencies, and error handling, and finally uses iterative reasoning to determine whether the implementation satisfies the requirement. Each result is reported as CONSISTENT, INCONSISTENT, or INSUFFICIENT, with supporting or missing evidence recorded explicitly. For suspicious inconsistent results, SpecField further attempts to generate executable validation tests.

We evaluate SpecField on real-world implementations of TLS/DTLS, MQTT, QUIC, and CoAP. SpecField produced 204 submitted conformance-deviation reports; maintainers confirmed or fixed 126, rejected 9 as false positives, and 69 remain pending. Two confirmed reports were assigned previously unknown CVEs. Under GPT-5.4 pricing, the average model cost is $1.122 per submitted report.

Our main contributions are:

- We formulate protocol conformance checking as a field-centric alignment problem between standards and implementations, and design SpecField. The framework extracts structured field rules from natural-language standards and aligns them with implementation code contexts to identify conformance deviations in implementations.
- We introduce an auditable reporting workflow that turns model-assisted analysis into maintainer-oriented defect reports and reproducible validation artifacts, reducing reliance on raw LLM verdicts and supporting manual confirmation.
- We evaluate SpecField on real-world implementations of TLS/DTLS, MQTT, QUIC, and CoAP, producing 204 submitted reports, 126 maintainer-confirmed or fixed reports, 9 rejected false positives, 69 pending reports, and two previously unknown CVEs.

II. Background

A. Field-Level Conformance

Protocol conformance asks whether an implementation satisfies the behavioral requirements stated by a standard. In natural-language standards, many requirements are attached to protocol fields: message fields, parameters, lengths, flags, and valid field combinations. We call this scope *field-level semantic conformance*.

Field-level conformance requires two links: mapping the standard's field and condition to implementation objects, and checking whether the relevant code path enforces the required behavior. This scope is narrower than full protocol verification, but it covers many practical deviations because parsing, validation, negotiation, and error handling are often organized around fields.

B. Normative Modalities

Normative modality determines whether a statement is treated as a reportable obligation. SpecField extracts *MUST* and *MUST NOT* requirements as primary conformance rules, tracks *SHOULD* requirements as lower-severity deviations, and ignores *MAY* statements because they describe optional behavior. The modality is stored with each rule and used when ranking reports.

III. Method

Figure 1 shows the SpecField pipeline. The system takes two inputs: a natural-language protocol standard and a project codebase. It then executes three analysis stages before producing outputs: (1) LLM-based rule extraction from the standard, (2) code-context extraction from the implementation, and (3) a reasoning-and-test agent that combines both streams of evidence. The output is not only a label, but a summary report, a bug report, and reproduction artifacts.

Standard-document analysis. Field requirements in protocol standards are often scattered across natural-language descriptions, state conditions, boundary constraints, and exception rules. SpecField first locates protocol variables and their associated constraints in the standard, and then extracts field-level rules from this material. This design avoids asking an LLM to directly judge defects from large blocks of standard text. Instead, it first derives well-scoped checking rules that provide the basis for subsequent code-conformance analysis.

Code-context extraction. The left-bottom input box represents implementation evidence: C files, headers, structures, enums, macros, functions, and call graphs. SpecField maps each standard variable to implementation-level identifiers and then extracts the code contexts needed to evaluate the corresponding rule. This produces the code evidence consumed by the reasoning agent.

Reasoning-and-test agent. The agent first performs consistency reasoning and may return *insufficient*, *consistent*, or *inconsistent*. Insufficient evidence triggers another code-context extraction round. For inconsistent or unresolved cases, the verification-test component generates a test plan, builds and runs the test, and judges the observed behavior.

Reports and artifacts. SpecField produces three classes of outputs: a summary report for manual triage, a defect report for maintainers, and validation artifacts for checking and reproduction. Because LLM judgments are not always reliable, simply reporting whether a rule is violated is insufficient for follow-up review and repair. SpecField therefore provides traceable evidence and reproducible artifacts to help analysts confirm issues faster and reduce the cost for maintainers to understand and fix defects.

IV. Design Details

A. Problem Definition

Given a protocol standard S and an implementation P, SpecField seeks to identify whether P satisfies a set of normative field-level requirements in S. A requirement is represented as a rule $r = \langle f, C, A, M, E \rangle$, where f is the constrained protocol field, C is the applicability condition, A is the expected action or constraint, M is the normative modality such as *MUST* or *SHOULD*, and E records source evidence. The checker returns a tuple $\langle y, E, U, T \rangle$: label $y \in \{\textsc{Consistent}, \textsc{Inconsistent}, \textsc{Insufficient}\}$, evidence E, uncertainty notes U, and optional validation-test artifact T.

SpecField does not rely on naming similarity between protocol implementations and standard text. Instead, it extracts a structured intermediate representation from the standard and maps that representation to relevant variables and code contexts in the implementation using a combination of semantic and syntactic evidence.

B. Field-Space Construction

Directly extracting rules from open-ended prose is unstable because standards mix design motivation, compatibility advice, examples, and normative requirements. SpecField therefore starts by constructing a *field space*: a set of protocol objects that subsequent rules may reference. Field candidates are collected from message formats, extension structures, parameter tables, and explicit field-definition paragraphs. For each field, SpecField records its canonical name, aliases, parent message or structure, type, allowed values when available, and source locations.

Document preprocessing. SpecField ingests RFC-style text, PDF, DOC, and TXT documents. To preserve paragraph structure and local semantic coherence, it first splits the document into paragraphs on blank lines and then greedily merges consecutive paragraphs into text blocks. Paragraphs are never split during this process. When adding the next paragraph would exceed a configurable maximum character threshold, the current block is committed and a new block is started from that paragraph. The default threshold is 6000 characters, and it can be adjusted via a command-line argument. This character-level threshold provides stable, tokenizer-independent block boundaries during preprocessing.

The field space serves two purposes. First, it gives the extractor a closed set of referents, reducing the chance that descriptive concepts such as "compatibility mode" are mistaken for checkable fields. Second, it preserves cross-document

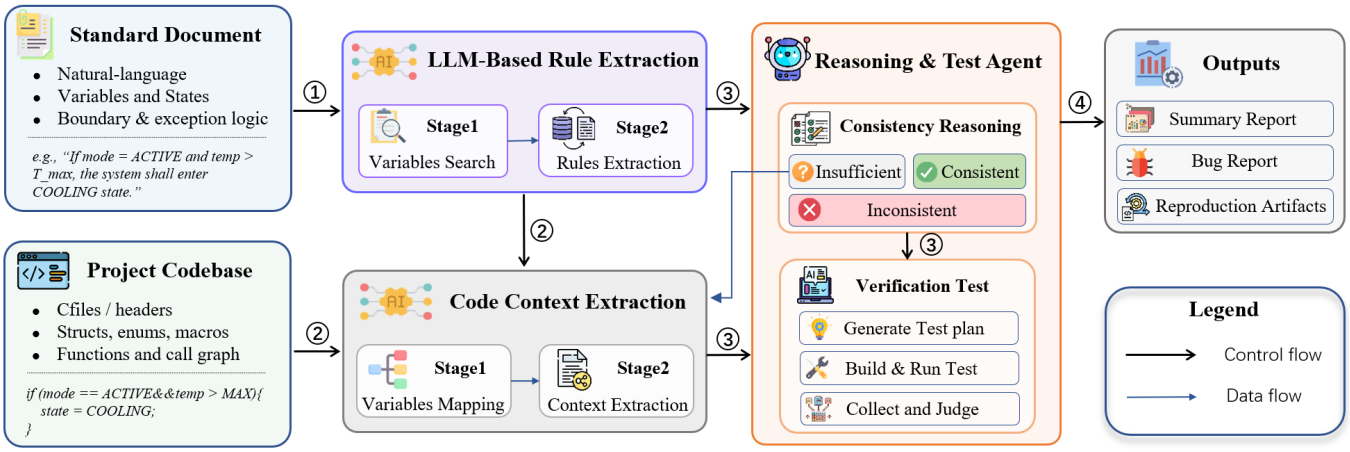

Fig. 1. Overview of SpecField. The original system design has four modules: standard-document analysis, LLM-based rule extraction, code-context extraction, and a reasoning-and-test agent that emits summary reports, bug reports, and reproduction artifacts.

identity: the same field may appear as a full path, short name, structure member, or contextual pronoun. During rule extraction, all references must resolve to one or more field-space entries. Ambiguous references are kept as uncertainty rather than silently collapsed.

Field identity and validation. SpecField treats two mentions as the same field if their parent context and semantic role are compatible, preventing generic names such as *length*, *type*, or *flags* from being merged across unrelated messages. Before rule extraction, it also validates the field space by marking entries with missing parent context, incompatible aliases, or only weak prose evidence as ambiguous or out of scope. This conservative policy favors precision over recall because an incorrect field merge can mislead all downstream alignment and reasoning.

C. Field-Centric Rule Extraction

After field-space construction, SpecField uses each standard section as the basic unit of analysis to extract field-centric rules from normative text. Specifically, for each section, the system provides the section text, the field-space entries relevant to that section, and a normative keyword lexicon built from RFC 2119-style language to the model. Within these constraints, the model identifies normative statements and resolves the protocol objects they mention to concrete entries in the field space. It then emits structured records under a fixed rule schema, including the constrained field, applicability condition, expected action or constraint, normative modality, and source evidence. *MAY*-level statements are filtered out of this rule set because they do not impose required behavior in our checking scope. The resulting schema is summarized in Table I.

Local scope and validation. This design confines rule extraction to local sections and a known field set, preventing open-ended inference over the full standard and reducing unsupported rule generation. The field space provides a traceable referential basis for protocol objects, so each rule can be linked back to specific fields and source text. For normative sentences that contain multiple constraints, SpecField splits them into multiple field-level rules, each corresponding to an independent checkable behavior such as field presence, value constraints, range checks, state updates, or rejection behavior.

TABLE I
RULE SCHEMA USED BY SPECFIELD.

Attribute	Description
Field	Constrained protocol field or parameter
Condition	Message type, state, phase, or field predicate
Action	Presence, value, derivation, relation, or rejection
Modality	MUST, MUST NOT, SHOULD, or equivalent phrase
Evidence	Source section, sentence span, and referenced text

After extraction, SpecField validates model outputs with a deterministic checker to ensure structural validity and provenance traceability. The checks include whether field references resolve to field-space entries, whether the source span exists, whether the action type belongs to a supported action class, and whether the rule duplicates an existing record. Outputs that fail validation are retried with a concise repair prompt; if they still fail, the candidate rule is discarded rather than turned into an unauditable rule.

The action families are intentionally close to implementation behavior: *assign*, *derive*, *select*, *require-present*, *require-absent*, *compare*, *range-check*, *state-update*, and *reject*. This design encourages a rule to map to observable code constructs such as assignments, conditionals, error returns, and state transitions. When a sentence contains multiple constraints, SpecField atomizes it into multiple rules so that each rule can be aligned and tested independently.

Rule normalization. Before alignment, rules are normalized to make equivalent requirements compare more consistently. Negated forms are rewritten into explicit actions, e.g., "MUST NOT accept x" becomes an expected *reject* action under condition x. Cross-references are expanded when

the referenced section contains a field-space entry. Multiple modalities in the same sentence are split into separate rules. The normalized rule remains linked to the original text so that reviewers can audit whether the transformation preserved the intended meaning.

Unsupported rules. Some normative statements are intentionally not converted into field-level rules. Examples include *MAY*-level statements, broad performance guidance, implementation advice without observable behavior, and cryptographic assumptions that require a proof rather than code-level checking. SpecField records these statements as *out-of-scope normative text*. Reporting out-of-scope text is useful: it prevents silent loss of requirements and makes the method's coverage explicit.

D. Syntax-Aware Rule-to-Code Alignment

The alignment stage maps a rule r to implementation variables and code contexts. It proceeds in three steps.

File ranking. SpecField first builds a file-level summary for each candidate C/C++ file using its path, function signatures, structure and enum definitions, comment summary, and hits on field aliases. An LLM-based ranker then takes the rule, field aliases, action class, and file summaries as input and estimates whether each file is likely to contain the relevant field representation or checking logic. The ranking considers field or alias matches, the relationship between file functionality and the rule action, and the relationship between the file's module and the target protocol object. The top-ranked files are deduplicated and passed to variable localization.

Variable localization. For the constrained field in the rule, SpecField first constructs a set of field aliases to bridge naming differences between standard field names and implementation identifiers. The system tokenizes field names, normalizes naming styles, and expands the alias set using common abbreviations, parent message names, and parent structure names. It then builds a C/C++ symbol index with Tree-sitter [14], covering variables, structure members, enum constants, macros, and function parameters. For each indexed entity, SpecField records its declaration locations, use locations, scope, and enclosing function.

On top of the symbol index, SpecField ranks candidate identifiers using name matches and syntactic usage patterns. Identifiers that exactly match a field alias or have high token overlap receive higher priority. The rank is further increased when the identifier appears in assignment, comparison, bounds checking, state update, or error-return logic relevant to the rule action. Conversely, identifiers with overly generic names, identifiers that appear only as temporary or loop variables, and candidates without relevant syntactic usage evidence are down-ranked. This stage outputs ranked implementation-variable candidates and their declaration and use contexts as anchors for subsequent code-context retrieval.

Context retrieval. For each candidate variable v, SpecField retrieves code snippets from three sources: exact textual hits, symbol scopes containing the hits, and semantic windows whose tokens overlap the rule. We use a fixed context-retrieval before ranking each snippet c before LLM semantic filtering. The score is not intended to be an optimal or learned ranking function. Instead, it instantiates an ordinal retrieval-signal hierarchy: exact field or alias occurrence should dominate variable-token overlap, variable-token overlap should dominate weak rule-token overlap, and the action-role bonus should favor snippets that manipulate the localized field in a way compatible with the rule action.

$$\begin{aligned} S_{\mathrm{ctx}}(c) = w_{\mathrm{exact}} \cdot \mathbb{I}_{\mathrm{exact}}(c) + w_{\mathrm{var}} \cdot |\mathcal{T}_c \cap \mathcal{T}_v| \\ + w_{\mathrm{rule}} \cdot |\mathcal{T}_c \cap \mathcal{T}_r| + b(c). \end{aligned} \tag{1}$$

where $\mathbb{I}_{\mathrm{exact}}$ denotes exact variable or alias occurrence, $\mathcal{T}_c$ is the code-token set, $\mathcal{T}_v$ is the variable-token set, $\mathcal{T}_r$ is the rule-token set, and $b(c)$ rewards action-relevant usage patterns such as initialization, validation, comparison, error handling, and configuration reads. By default, $w_{\mathrm{exact}} = 5$, $w_{\mathrm{var}} = 0.9$, and $w_{\mathrm{rule}} = 0.28$. Snippets without exact variable or alias occurrence receive a score cap to avoid weak semantic matches dominating direct code matches.

The top snippets are then sent to an LLM semantic filter. The filter sees only the rule and the ranked snippets, and selects the code contexts most relevant to the rule. For each selected context, it explains the correspondence between the snippet and the rule and returns a confidence score. Finally, the model collects code contexts related to field representations, checking logic, and error-handling paths for subsequent consistency reasoning.

The constants in Equation (1) are therefore not a claim of universal optimality. They encode a stable ordering among retrieval signals. Exact field or alias occurrence is treated as the strongest signal because it directly links the snippet to the localized implementation representation. Variable-token overlap captures renamed or split representations. Rule-token overlap is weaker because textual similarity to the standard often retrieves comments or unrelated helper functions. The action-role bonus rewards snippets where the candidate field participates in parsing, assignment, comparison, rejection, error propagation, configuration reads, or state updates. We select the default constants once on held-out sections and do not tune them using bug-finding outcomes.

Why not pure retrieval? Pure semantic retrieval often retrieves a function whose comments resemble the standard but whose variables are unrelated to the constrained field. Pure exact matching has the opposite problem: it misses implementations that rename fields or distribute a field across parser state and connection state. SpecField uses syntax-aware retrieval signals as the meeting point. A snippet is strongest when it both mentions a plausible representation of the field and uses that representation in a role consistent with the rule action.

E. Consistency Reasoning

After obtaining code contexts relevant to a rule, SpecField enters the consistency-reasoning stage. This stage confines reasoning to the current evidence bundle rather than asking the model to inspect the entire codebase. The bundle consists

Algorithm 1: Evidence-driven consistency reasoning.

Input: Rule r, implementation index I, maximum rounds K
Output: Consistency tuple $\langle y, E, U, T \rangle$

$C_0 \leftarrow \text{AlignRuleToCode}(r, I)$
for $i \leftarrow 0$ **to** K **do**
  $o_i \leftarrow \text{Reason}(r, C_i)$
  if $o_i.y \neq \text{Insufficient}$ **then**
    return o_i
  $q_i \leftarrow \text{ContextRequest}(o_i.U)$
  $C_{i+1} \leftarrow C_i \cup \text{RetrieveAdditionalContext}(q_i, I)$
  if $C_{i+1} = C_i$ **then**
    return o_i
return o_K

of the field rule, standard source span, localized implementation variables, candidate code snippets, call relationships, and configuration conditions. Based on this evidence, the reasoning agent decides whether the observed implementation behavior satisfies the normative obligation in the rule and outputs a structured result $\langle y, E, U, T \rangle$. Here, y is the judgment label and takes one of CONSISTENT, INCONSISTENT, or INSUFFICIENT; E records the standard spans and code evidence supporting the judgment; U records uncertainty in the current judgment and the missing context; and T denotes validation-test artifacts, which are usually empty during static reasoning and are populated later by the validation stage.

The reasoning process is evidence-bounded. The model may reason only from the standard spans, code snippets, and call relationships in the evidence bundle, and it may not assume behavior along code paths that have not been retrieved. If the evidence shows that the implementation explicitly performs the check or handling required by the rule, the result is CONSISTENT. If the evidence shows behavior that contradicts the normative requirement, the result is INCONSISTENT. If the current evidence does not cover a key path, such as a caller, macro definition, configuration branch, validation function, or error-propagation path, the result is INSUFFICIENT. In that case, U records not only the reason for uncertainty but also the context request needed for the next retrieval round, such as finding callers of a validation function, the definition of a macro, the propagation path of an error code, or the later uses of a field assignment.

The system then supplements the code context according to the requests in U and merges the new evidence into the bundle for another reasoning round. Algorithm 1 describes this process. The loop terminates when the reasoning agent returns a determinate label, when retrieval cannot provide new useful context, or when the preset round limit is reached. Treating INSUFFICIENT as an explicit result prevents SpecField from mistaking lack of evidence for implementation inconsistency. It also gives every additional retrieval step a concrete target, improving the traceability and reliability of the final consistency judgment.

F. Validation-Test Agent

Candidate reports labeled INCONSISTENT by static reasoning may still be false positives. A check that appears to be missing in a local snippet may be enforced by an upper-level caller or state-machine path. Conversely, macro definitions, build options, and conditional compilation can change which code paths are active, making a judgment based only on local static evidence incomplete or incorrect. SpecField therefore introduces a validation-test agent for potential inconsistencies, using executable behavior to further validate the static reasoning result.

The validation-test agent receives the field rule, defect hypothesis, and relevant code snippets, and generates a rule-directed test plan. The plan attempts to construct two kinds of counterexamples: inputs that violate the standard requirement but are accepted by the implementation, and inputs that satisfy the standard requirement but are rejected by the implementation. The agent then prepares the runtime environment, selects required build options, generates or adapts a test harness, executes the test, and collects runtime logs.

SpecField confirms a report as inconsistent only when the test stably reaches the target path and the observed behavior contradicts the field rule. If the test cannot reach the relevant path, or if the result is insufficient to support the conclusion, the system records the unmet validation conditions rather than promoting the static defect hypothesis to a confirmed issue. This conservative policy ensures that confirmed results are grounded in reproducible runtime evidence.

The validation runner supports three execution modes: extending an existing unit test, constructing a small harness around a parser or validation function, and driving a command-line implementation with generated inputs. For each run, the system records compiler options, environment variables, execution commands, exit status, stdout, stderr, and relevant logs. Build failures, unreachable paths, and insufficient observations are classified as unmet validation conditions and are not counted as confirmed inconsistencies.

After validation, SpecField generates a structured validation report and reproduction artifacts. The report includes the field rule, related standard text, key code snippets, inconsistency rationale, test input, build configuration, execution command, observed result, and log summary. For reproducible inconsistencies, the report provides reproduction instructions and script commands so maintainers can rerun the test under the same code version and build conditions. For unconfirmed results, the report explicitly lists the failure reason, such as build failure, unreachable target path, missing test entry point, or insufficient observation signal. In this way, validation is not only a confirmation step but also a traceable evidence chain for manual review and maintainer communication.

V. EVALUATION

We evaluate SpecField on real protocol implementations and answer five research questions:

TABLE II
PROTOCOL IMPLEMENTATIONS USED FOR EVALUATION.

Subject	Version	LoC	Protocol	Specification
OpenSSL	11b7b6e	1456.3K	TLS 1.3	RFC 8446
wolfSSL	1d363f3	520.4K	TLS 1.3	RFC 8446
Mbed TLS	0fe989b	310.8K	TLS 1.3	RFC 8446
openHiTLS	3d295814	480.2K	TLS 1.3	RFC 8446
Mosquitto	99fa50f	180.6K	MQTT 5.0	MQTT 5.0
Mosquitto	99fa50f	180.6K	MQTT 3.1.1	MQTT 3.1.1
wolfMQTT	88d37ed	41.5K	MQTT 3.1.1	MQTT 3.1.1
quiche	851533c	92.3K	QUIC	RFC 9000
ngtcp2	4af1c77	130.1K	QUIC	RFC 9000
libcoap	7bc9a41	115.0K	CoAP	RFC 7252
BoringSSL	7f4d9a2	760.4K	DTLS 1.3	RFC 9147

- **RQ1: Field-centric rule discovery.** In a pooled open-world setting, how accurately and stably does SpecField discover field-level rules for each protocol standard?
- **RQ2: Rule-to-code alignment.** How accurately does SpecField map a standard rule to implementation variables and contexts?
- **RQ3: Bug finding.** In a pooled open-world setting, how many conformance-deviation reports does SpecField discover, and how many are confirmed, fixed, rejected, pending, or assigned CVEs?
- **RQ4: Cost.** What is the LLM cost of generating submitted reports under GPT-5.4 pricing and across model providers?
- **RQ5: Ablation.** Which system components are responsible for the observed performance?

A. Subjects and Baselines

Table II lists the evaluation subjects used in this study. To ensure a fair comparison across different implementations, we require *all implementations of the same protocol version to share the same rule set* extracted from the corresponding standard. For example, all TLS 1.3 implementations are checked against the same 312 rules extracted from RFC 8446 [2], and the two QUIC implementations are checked against the same 286 rules extracted from RFC 9000 [5]. Specifically, the TLS/DTLS subjects include OpenSSL [15], wolfSSL [16], Mbed TLS [17], openHiTLS, and BoringSSL [18]; the QUIC subjects include quiche [19] and ngtcp2 [20]; the MQTT subjects include Mosquitto [21] and wolfMQTT [22]; and the CoAP subject is libcoap [23].

We compare SpecField against four baselines under the same subject versions, rule budgets, model family, temperature setting, and maximum reasoning rounds. **Direct Agent** receives the relevant standard section plus the top keyword-retrieved code snippets and is asked to produce conformance reports without field-space construction. **LLM+RAG** uses the same prompt schema but retrieves code by embedding similarity over function-level chunks. **Static-only** keeps SpecField's rule and code indexes but replaces LLM semantic filtering and reasoning with deterministic alias, token-overlap, and error-path heuristics. **w/o Test Agent** runs the full pipeline but does not attempt executable validation. Each baseline uses the same top-15 file, top-8 variable, top-12 snippet, and up to three evidence-completion rounds unless that component is absent by design; prompt templates and raw outputs are included in the artifact.

B. Metrics and Ground Truth

For rule extraction, we evaluate field-rule discovery in a pooled open-world setting. To avoid bias toward any single LLM decoding run, we independently run the full extractor three times for each extraction configuration. Candidate rules from the three runs are first pooled and deduplicated by protocol version, source span, constrained field, applicability condition, expected action, and modality, forming the candidate pool for that configuration.

We then compare the pool against the manually annotated benchmark of field-level obligations. This benchmark was annotated by researchers from the TLS 1.3 and MQTT 3.1.1 standards and contains 430 field-level obligations; each obligation includes the source span, constrained field, condition, expected behavior, and modality. If a candidate rule, after normalization, matches a gold obligation in field, condition, action, and source semantics, it is counted as correctly discovered. If a candidate rule matches no gold obligation, or corresponds to unsupported normative text, erroneous splitting, or hallucinated requirements, it is counted as a false discovery. If a gold obligation is not discovered in any of the three runs for that configuration, it is counted as missed. The table values are therefore obtained by matching the pooled candidate rules against the 430 gold obligations.

To evaluate code-context extraction, we use Var@1, Var@3, Ctx@1, Ctx@3, MRR, and average token consumption. We stratify-sample 312 rule-implementation pairs by protocol and implementation. Two authors with protocol and systems experience independently annotate the correct implementation variable in each sample and the minimal code context needed to determine the rule's behavior. Disagreements are resolved by joint review of the standard span, callers, validation helpers, and tests.

Here, Var@1 indicates whether the top-ranked candidate implementation variable is the gold variable; Var@3 indicates whether the gold variable appears among the top three candidates. Ctx@1 indicates whether the top-ranked code context contains the key evidence required to judge the rule, such as a field representation, a relevant check, error propagation, or an error-handling path; Ctx@3 indicates whether at least one of the top three code contexts contains such evidence. When evidence is spread across multiple functions, a top-k result is counted as a hit as long as the context bundle contains both the decisive callee and caller. MRR measures the average position of the correct result in the ranking list by taking the reciprocal rank of the first correct candidate for each sample and averaging over all samples. For example, a correct result at rank 1 scores 1, at rank 2 scores 1/2, and at rank 3 scores 1/3; if no correct result is found, the sample scores 0. Average token

consumption measures the number of LLM tokens consumed per rule-implementation pair during alignment.

To evaluate the vulnerability-finding capability of different models, we conduct a model-level comparison in a pooled open-world setting. Specifically, we run the full pipeline with four model families, namely GPT, Claude, DeepSeek, and Qwen, and repeat the experiment three times independently for each model. Candidate reports from all twelve runs are first pooled and deduplicated by protocol version, implementation, normative rule, and observed behavioral deviation, so that the same issue is not counted multiple times. Security experts then manually adjudicate the deduplicated reports according to predefined correctness criteria and retain the true conformance deviations for each implementation. For a given model, a report is counted as discovered if the model finds it in at least one of its three runs. Based on this shared adjudicated report pool, we report the number of true reports and false positives discovered by each model, and define pooled coverage as the fraction of all adjudicated true reports found by that model. Pending reports are excluded from the precision denominator because their final status depends on maintainer response latency. Meanwhile, we do not claim that this metric represents whole-corpus recall.

A submitted report is a deduplicated candidate issue with a standard span, normalized rule, code evidence, and reproduction material when available. A confirmed fixed report is a submitted report that the maintainer fixed or explicitly confirmed as fixed. A false positive is a submitted report rejected by maintainer feedback or expert adjudication. A pending report has not yet received enough external signal for final adjudication. CVEs are counted only for previously unknown vulnerabilities assigned CVE identifiers after disclosure.

Our precision denominator is the adjudicated subset, not all submitted reports: $\mathrm{P}_{adj} = \mathrm{Confirmed}/(\mathrm{Confirmed} + \mathrm{FP})$.

C. RQ1: Field-Centric Rule Discovery

Table III evaluates field-rule discovery on the manually annotated TLS 1.3 and MQTT 3.1.1 benchmark. For each extraction configuration, we independently run the extractor three times, then pool and deduplicate the candidate rules from the three runs before matching them against the 430 gold field-level obligations. We compare against three baselines: 0-shot prompt directly extracts rules from the standard text; 2-shot prompt adds a few examples to guide the model output; and Schema-only uses the same output schema and validators as SpecField, but does not construct a field space. All methods use the same standard chunking and model settings.

SpecField achieves the highest precision, coverage, and F1. The main reason is that the field space provides clear protocol-object boundaries for rule extraction. On the one hand, it reduces cases where descriptive text, compatibility notes, or generic concepts are mistakenly extracted as field rules. On the other hand, it helps the model resolve aliases, contextual references, and cross-paragraph field constraints, thereby reducing missed extractions. Therefore, SpecField's larger downstream rule set is not caused by over-fragmenting standard text, but by more accurate discovery of field-level obligations.

TABLE III
FIELD-RULE DISCOVERY PERFORMANCE.

Method	Extract.	TP	FP	FN	Prec.	Cov.	F1
0-shot prompt	319	263	56	167	0.82	0.61	0.70
2-shot prompt	384	323	61	107	0.84	0.75	0.79
Schema-only	402	347	55	83	0.86	0.81	0.83
SpecField	439	407	32	23	0.93	0.95	0.94

TABLE IV
EFFECTIVENESS OF RULE-TO-CODE ALIGNMENT.

Method	Var@1	Var@3	Ctx@1	Ctx@3	MRR	Tok.
Exact Match	0.34	0.48	0.29	0.43	0.41	0.9K
BM25	0.41	0.56	0.36	0.51	0.48	1.3K
Embedding	0.47	0.64	0.43	0.59	0.55	2.5K
LLM Direct	0.72	0.84	0.69	0.88	0.78	7.1K
SpecField	0.79	0.91	0.74	0.87	0.84	5.2K

D. RQ2: Rule-to-Code Alignment

Table IV shows that field-aware localization substantially improves rule-to-code alignment. Compared with Exact Match, BM25, Embedding, and LLM Direct, SpecField achieves the best overall performance in both variable recovery and context recovery. Exact Match relies on lexical overlap, BM25 uses sparse lexical retrieval, Embedding uses dense semantic retrieval, and LLM Direct feeds retrieved code directly to the LLM without field-aware localization.

SpecField achieves 0.79 Var@1 and 0.91 Var@3, indicating that the correct implementation variable is usually ranked near the top. It also achieves 0.74 Ctx@1 and 0.87 Ctx@3, showing that the retrieved contexts more often contain the decisive evidence needed for checking a rule, such as validation logic, error propagation, or state updates. This improvement is important because many false positives arise when retrieval returns nearby but semantically irrelevant functions. Meanwhile, SpecField consumes 5.2K tokens on average, compared with 7.1K tokens for LLM Direct, suggesting that field-aware localization improves alignment quality while keeping the evidence bundle more compact.

a) Context-retrieval prior ablation.: The coefficients in Equation (1) are fixed retrieval priors rather than project-specific learned weights. Table V ablates the retrieval signals and varies their weights to test whether Equation (1) depends on fragile constants. Here, variable-token overlap means the overlap between a candidate snippet and tokens from the localized implementation variable; rule-token overlap means the overlap between a candidate snippet and tokens from the field-level rule; and the action-role bonus $b(c)$ rewards snippets where the candidate field participates in validation, comparison, assignment, error handling, or state updates. The results show that removing exact-field matches causes the largest performance drop, while removing weaker terms leads

TABLE V
SENSITIVITY OF RULE-TO-CODE ALIGNMENT SIGNALS.

Setting	w_{exact}	w_{var}	w_{rule}	**Var@1**	**Ctx@1**	**MRR**
Default	5.0	0.90	0.28	0.79	0.74	0.84
No exact	0.0	0.90	0.28	0.70	0.63	0.75
No var. tok.	5.0	0.00	0.28	0.73	0.66	0.78
No rule tok.	5.0	0.90	0.00	0.77	0.72	0.82
No action	5.0	0.90	0.28	0.75	0.69	0.80
$w_{\text{exact}} \times 0.5$	2.5	0.90	0.28	0.78	0.73	0.83
$w_{\text{exact}} \times 2$	10.0	0.90	0.28	0.79	0.74	0.84
$w_{\text{var}} \times 0.5$	5.0	0.45	0.28	0.78	0.73	0.83
$w_{\text{var}} \times 2$	5.0	1.80	0.28	0.79	0.74	0.84
$w_{\text{rule}} \times 0.5$	5.0	0.90	0.14	0.78	0.73	0.83
$w_{\text{rule}} \times 2$	5.0	0.90	0.56	0.78	0.73	0.83
Uniform weights	1.0	1.00	1.00	0.74	0.68	0.79

TABLE VI
STATUS OF SUBMITTED CONFORMANCE-DEVIATION REPORTS AND MAINTAINER OUTCOMES. #R DENOTES THE NUMBER OF RULES USED FOR EACH PROTOCOL VERSION. SUB. IS THE NUMBER OF REPORTS SUBMITTED TO MAINTAINERS, CONF. IS THE NUMBER OF REPORTS CONFIRMED OR FIXED BY MAINTAINERS, FP IS THE NUMBER OF REJECTED FALSE POSITIVES, AND PEND. IS THE NUMBER OF REPORTS STILL AWAITING FINAL MAINTAINER RESPONSE.

Subject	**Protocol**	**#R**	**Sub.**	**Conf.**	**FP**	**Pend.**
OpenSSL	TLS 1.3	312	14	9	1	4
wolfSSL	TLS 1.3	312	23	15	1	7
Mbed TLS	TLS 1.3	312	17	11	1	5
openHiTLS	TLS 1.3	312	19	12	1	6
Mosquitto	MQTT 5.0	164	21	13	1	7
Mosquitto	MQTT 3.1.1	127	16	10	0	6
wolfMQTT	MQTT 3.1.1	127	22	14	1	7
quiche	QUIC	286	24	15	1	8
ngtcp2	QUIC	286	18	11	1	6
libcoap	CoAP	143	18	11	1	6
BoringSSL	DTLS 1.3	198	12	5	0	7
Total	–	–	204	126	9	69

to more gradual degradation in alignment quality. When the exact-field, variable-token, and rule-token weights are scaled within a factor of two while preserving their relative order, the alignment metrics remain stable. In contrast, the largest degradation appears when the retrieval-signal hierarchy is broken, especially when exact-field matches are removed, $b(c)$ is disabled, or all weights are assigned uniform values. These results suggest that SpecField's alignment quality mainly comes from the retrieval-signal hierarchy and syntax-aware features, rather than from a narrowly tuned set of constants. In Table V, "var. tok." and "rule tok." denote variable-token and rule-token overlap, respectively.

E. RQ3: Bug Finding Results

Table VI summarizes the status of reports that were submitted to repository maintainers. The table does not count all intermediate candidate findings produced by the pipeline. Instead, it reports the final deduplicated submissions and their maintainer outcomes. In total, SpecField submitted 204 conformance-deviation reports. Maintainers have confirmed or fixed 126 of them, rejected 9 as false positives, and 69 remain pending. Two confirmed reports correspond to previously unknown vulnerabilities and were assigned CVE identifiers after disclosure.

To evaluate the bug-finding capability of different model families, Table VII compare their report outcomes under the same pipeline setting. We fix all non-model components to ensure a fair comparison across models. Specifically, each model uses the same extracted rules, code-retrieval budget, validators, adjudication protocol, and test-agent settings. The code-retrieval budget includes the number of candidate files, candidate variables, and code snippets that can be retrieved for each rule, as well as the maximum number of evidence-completion rounds. Therefore, different models are given rule sets and code evidence of the same scale, and the differences mainly come from the models' semantic understanding, code reasoning, and report-generation capabilities. Prompt differences are limited to model-specific formatting constraints needed to obtain schema-valid JSON outputs. The results show that GPT-5.4 produces the largest number of maintainer-confirmed fixes. In contrast, smaller or less code-specialized models produce more false-positive reports. Provider-level cost differences are analyzed separately in RQ4.

TABLE VII

PER-MODEL COMPARISON ON CONFORMANCE INCONSISTENCY DETECTION IN THE POOLED OPEN-WORLD SETTING. THE TWELVE RUNS ARE POOLED AND DEDUPLICATED BY PROTOCOL VERSION, IMPLEMENTATION, NORMATIVE RULE, AND OBSERVED BEHAVIORAL DEVIATION. ADJ.P EXCLUDES PENDING REPORTS FROM THE DENOMINATOR.

Protocol	**#R**	**GPT-5.4**				**DeepSeek**				**Claude**				**Qwen**				**Avg. Adj.P**
		Sub.	**Conf.**	**FP**	**Adj.P**	**Sub.**	**Conf.**	**FP**	**Adj.P**	**Sub.**	**Conf.**	**FP**	**Adj.P**	**Sub.**	**Conf.**	**FP**	**Adj.P**	
TLS 1.3	312	73	47	4	0.92	62	37	7	0.84	68	42	6	0.88	56	31	7	0.82	0.86
QUIC	286	42	26	2	0.93	36	21	4	0.84	39	24	4	0.86	33	18	4	0.82	0.86
MQTT 5.0	164	21	13	1	0.93	18	10	2	0.83	19	12	2	0.86	17	9	2	0.82	0.86
MQTT 3.1.1	127	38	24	1	0.96	32	19	3	0.86	34	21	3	0.88	28	16	4	0.80	0.88
CoAP	143	18	11	1	0.92	16	10	1	0.91	17	10	2	0.83	15	9	2	0.82	0.87
DTLS 1.3	198	12	5	0	1.00	12	5	1	0.83	11	5	2	0.71	10	4	2	0.67	0.80
Total	1230	204	126	9	-	176	102	18	-	188	114	19	-	159	87	21	-	-

F. RQ4: Cost Analysis

To evaluate the model cost of SpecField, we use GPT-5.4 pricing as the cost model. Table VIII breaks down the average cost per report by pipeline stage, including rule extraction, code-context extraction, consistency reasoning, and the verification-test agent. The largest cost comes from consistency reasoning, which costs $0.431 per report on average and accounts for 38.4% of the total cost. This is because a suspicious finding usually cannot be judged in a single round. Instead, it often requires multiple evidence-completion rounds, such as retrieving callers, macro definitions, error-propagation paths, or configuration branches. Code-context extraction is the second-largest cost component, accounting for 25.3%, because the system needs to semantically filter candidate files, variables, and code snippets before reasoning. The verification-test agent accounts for 20.6%, mainly due to test-plan generation, harness adaptation, and log interpretation. In contrast, rule extraction accounts for only 15.7%, because the rule set for the same protocol version can be reused across multiple implementations and its cost is therefore amortized.

Figure 2 further compares the same stage-wise cost breakdown across the four model families used in Table VII. Claude has the highest estimated total cost, followed by GPT-5.4, while DeepSeek and Qwen are about 35% cheaper than GPT-5.4. These results show a trade-off between model cost and bug-finding capability: lower-cost models reduce analysis expense, but, as shown above, they generally produce fewer maintainer-confirmed fixes or more false positives. In practice, one possible deployment strategy is to use lower-cost models for initial screening and then use stronger models to recheck high-risk rules or suspicious reports.

G. RQ5: Ablation Study

Taking GPT-5.4 as the example, Table IX shows how different system components affect discovery capability and result quality. Direct Agent and LLM+RAG cover 70.1% and 64.7% of the reports, respectively, showing that standard snippets and generic code retrieval alone do not reliably obtain the implementation evidence needed for rule checking. In contrast, Static-only keeps the rule representation and code

TABLE VIII

AVERAGE GPT-5.4 MODEL COST PER SUBMITTED REPORT.

Stage	**Cost / Report**	**Share**
Rule extraction	$0.176	15.7%
Code-context extraction	$0.284	25.3%
Consistency reasoning	$0.431	38.4%
Verification-test agent	$0.231	20.6%
Total	$1.122	100.0%

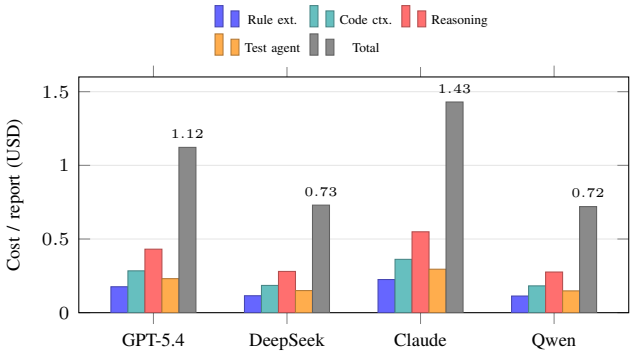

Fig. 2. Average model cost per submitted report by model family and pipeline stage. The GPT-5.4 bars correspond to Table VIII

indexes, increasing coverage to 83.3%. This result indicates that field-space construction and field-level code localization substantially expand the discoverable range. However, Static-only replaces LLM semantic filtering and reasoning with deterministic alias matching, token overlap, and error-path heuristics. Its false positives therefore rise to 78 and its adjusted precision drops to 0.60, showing that field localization helps the system find more potential evidence but is still insufficient for renamed fields, cross-function checks, and indirect error propagation.

Removing iterative reasoning reduces coverage to 78.9%. This result shows that the INSUFFICIENT state and the evidence completion it triggers are not only a conservative way to record uncertainty, but also directly affect discovery capability. When the current evidence lacks callers, macro

TABLE IX
ABLATION RESULTS ON THE ADJUDICATED REPORT SET. COV. DENOTES THE PERCENTAGE OF THE 204 REPORTS DISCOVERED BY EACH VARIANT, NOT WHOLE-CORPUS RECALL.

Configuration	Sub.	Conf.	FP	Pend.	Adj.P	Cov. (%)
Direct Agent	143	72	24	47	0.75	70.1
LLM+RAG	132	73	9	50	0.89	64.7
Static-only	170	116	78	54	0.60	83.3
w/o Ite Rea	161	101	15	60	0.87	78.9
w/o Test Agent	190	126	33	64	0.79	93.1
Forced binary	175	111	63	64	0.64	85.8
Full System	195	126	19	69	0.87	95.6

definitions, validation helpers, or error-propagation paths, the system must continue retrieving and completing context before a potential deviation can be submitted as a report. The Forced-binary result further supports this point: forcing low-evidence samples into either consistent or inconsistent verdicts covers only 85.8% of the reports and increases false positives to 63, showing that explicitly preserving INSUFFICIENT is more reliable than premature binary adjudication. Without the test agent, the system still reaches 93.1% coverage, but false positives increase to 33, indicating that runtime validation mainly filters and confirms high-risk reports rather than simply increasing the number of candidates. The full system combines field-space construction, semantic reasoning, INSUFFICIENT-driven evidence completion, and test validation, reaching the highest coverage of 95.6% while maintaining an adjusted precision of 0.87.

H. Case Studies

Case 1: A TLS implementation accepted an invalid negotiated field. SpecField extracted a MUST-level field constraint from RFC 8446 and mapped it to a parser state variable in a TLS implementation. The first reasoning round found only assignment logic, then requested the caller that validates the parsed value. The resulting report included a minimal transcript and was fixed by the maintainer.

Case 2: A QUIC transport-parameter bug became a 0-day CVE. SpecField identified a missing rejection condition for a boundary value in a QUIC transport parameter. The test agent generated an input that violated the standard but was accepted by the implementation. The issue was disclosed and assigned a CVE after confirmation.

Case 3: MQTT version-specific rules prevent false positives. MQTT 3.1.1 and MQTT 5.0 have different rule sets. Reusing the wrong rule set would cause false reports against Mosquitto. SpecField avoids this by extracting rules per protocol version and reusing that set only across implementations of the same version; the same pooled discovery logic then applies across the three runs of each model family.

I. Failure Analysis

The unresolved and rejected reports fall into three categories. First, some pending reports are awaiting maintainer triage even though they include a reproduction artifact. Second, some pending reports require multi-peer integration tests that our current validation agent cannot fully automate. Third, some rejected reports involve intentional compatibility behavior or SHOULD-level rules that maintainers do not consider bugs. These cases motivate stronger severity ranking and more protocol-specific test harnesses.

a) Reviewer-verifiable artifacts.: Our evaluation is designed to be reviewer-verifiable without de-anonymizing the authors. Each deduplicated report is assigned a blinded identifier and linked to: (i) the normative standard span, (ii) the normalized field rule, (iii) the implementation evidence category, (iv) the validation status, and (v) the maintainer/adjudication outcome category. Public identifiers are withheld only when they would compromise double-blind review. The artifact contains scripts that reproduce all aggregate tables from the blinded report index. All code, scripts, and blinded reports are included in an anonymous GitHub artifact repository submitted with the paper: https://anonymous.4open.science/r/SpecField-Artifact.

VI. DISCUSSION AND LIMITATIONS

Checking scope. SpecField primarily targets requirements that can be mapped to protocol fields, trigger conditions, and implementation behavior. It is well suited for checking local conformance deviations in parsing, validation, negotiation, and error handling. For global properties such as cross-session behavior or long-term state evolution, SpecField can only provide limited approximations and cannot offer completeness or correctness guarantees; strict conclusions still require separate modeling and verification.

Natural-language ambiguity. Ambiguity in standard text can affect rule extraction and conformance judgment. For example, a requirement such as ”reject malformed messages” may not clearly specify the relevant field combinations, compatibility exceptions, or error-handling paths. In such cases, SpecField may misclassify reasonable compatibility behavior, or it may miss real defects. Therefore, for rules with unclear semantic boundaries or rules that depend on implementation context, the final conclusion still requires expert interpretation and maintainer feedback. Our artifact preserves the source text, reasoning evidence, and uncertainty notes to support subsequent review.

LLM dependence. SpecField uses LLMs for extraction, ranking, semantic filtering, reasoning, and test planning. The framework constrains these calls with schemas, field spaces, syntax evidence, and deterministic validators, but it does not eliminate model error. We mitigate this risk by retaining uncertainty, using low-temperature decoding, validating outputs, and requiring executable evidence for confirmed reports.

Source availability. The current prototype relies on source code to locate variables, call relationships, and validation logic, and therefore does not support closed-source implementations. For implementations produced by code generators, or implementations that encapsulate substantial parsing, validation, and error-handling logic in macros, semantic cues in the

source code may be unstable or incomplete, which can affect the quality of specification-to-implementation alignment.

Language front-end. The current prototype mainly supports C/C++ protocol implementations. It combines syntactic analysis with LLM-assisted filtering to extract code context related to field accesses, conditional checks, and error handling from source code. For other programming languages, the system still requires corresponding language front-ends that provide equivalent code-analysis capabilities. In the future, we plan to extend this capability by integrating front-end analysis components for additional languages.

Result adjudication. Protocol conformance judgments cannot always be directly determined by the authors, because standards may be ambiguous and implementations may adopt different behaviors for compatibility. Therefore, system outputs should not be treated as final conclusions by themselves. We adjudicate reports as valid issues, false positives, or pending based on maintainer feedback, independent review, and reproducible tests. This also means that some results depend on external feedback and follow-up validation, and the current statistics only reflect the adjudication status at the time of evaluation.

Cost and scalability. Multi-round reasoning and validation are more expensive than one-shot prompting. SpecField reduces cost by caching standard-extraction results, compressing code contexts, and invoking validation only for suspicious or unresolved samples. Deployment in large protocol ecosystems still requires prioritization; one practical strategy is to first analyze check targets with high security relevance, dense historical defects, or strong rule constraints.

Threats to validity. Our evaluation may be affected by manual adjudication and model stability. First, manually labeled ground truth may be influenced by reviewers' interpretations of ambiguous standards. Second, LLM nondeterminism may change extraction or reasoning results across runs. We mitigate these threats by using two-author adjudication, preserving raw evidence, separating pending reports from confirmed fixes, and measuring variance across repeated GPT-5.4 runs and different model families.

VII. Related Work

Specification-guided protocol analysis. Recent protocol-analysis systems increasingly use standard documents to guide implementation checking. RIBDetector extracts RFC-derived rules to identify implementation inconsistencies [10]; ParCleanse uses LLMs to translate RFC documents into protocol message specifications that serve as traceable quasi-oracles for parser validation [11]; RFCAudit builds semantic indexes over protocol code and uses an LLM agent to retrieve implementation context for RFC-related defect detection [12]; and ProtocolGuard combines LLM-guided program slicing with assertion-guided fuzzing to detect protocol non-compliance [13]. Other recent systems have similar goals: iPanda uses retrieval, LLM agents, and execution feedback to generate conformance tests [24], and AUTOSPEC synthesizes formal protocol specifications from natural-language documents and implementation feedback to support test generation [25]. These systems also use protocol standards to guide analysis, but their outputs are usually rule matching results, parser oracles, retrieved code snippets, or dynamic counterexamples. The key difference of SpecField is that it represents each check target as a field-level conformance claim and requires the claim to bind the standard source, field semantics, applicability condition, implementation evidence, and evidence-sufficiency state. This allows SpecField to trace a suspicious behavior to a concrete consistent or inconsistent conclusion about a field requirement, rather than only reporting standard-related code snippets or anomalous executions.

Protocol testing. Traditional conformance suites and protocol fuzzers provide strong executable evidence, but their oracles are usually hand-written, inferred from observed behavior, or tied to protocol-specific testing frameworks. TLS-Attacker and TLS-Anvil generate targeted TLS tests [26], [27]; protocol state fuzzing and automata-based analyses expose deviations in TLS/DTLS state machines [28]–[31]; and monitor-based testing encodes protocol requirements as external monitors and uses symbolic execution to find DTLS and QUIC deviations [32]. QUIC conformance studies also show that independent implementations can drift, regress, or diverge as specifications evolve [33]. Certificate-validation studies use adversarial, differential, symbolic, and RFC-guided inputs to test SSL/TLS implementations [34]–[38]. General protocol fuzzers such as AFLNet and StateAFL improve exploration of stateful network protocols [39]–[42]. These methods are effective at exploring protocol implementations and finding defects, especially when test targets, input-generation strategies, or oracles are already available. SpecField is complementary to them: traditional testing and fuzzing emphasize behavior triggering and defect discovery, whereas SpecField starts from natural-language standards, aligns field-level requirements with implementation code, and determines whether the implementation deviates from the standard.

Formal verification. Formal methods have been widely used for protocol security analysis and implementation verification. For example, prior TLS 1.3 work uses ProVerif and CryptoVerif to model core protocol logic and verify security properties [43]–[45]. Such methods can provide strong guarantees, but they typically rely on precisely defined abstract models or reference specifications, rather than directly addressing large-scale conformance checking between natural-language standards and existing code implementations.

Other work more directly checks conformance between protocol standards and implementations. Continuous formal verification of Amazon s2n uses Cryptol and SAW to prove equivalence between a TLS implementation and high-level specifications, and integrates verification into the development process [46]–[48]. Work on the OpenSSL TLS 1.3 handshake state machine constructs Cryptol models from both the RFC and the OpenSSL implementation, and compares state-transition sequences to check equivalence between the standard and the implementation [49]. SYMNV uses symbolic execution to generate test inputs and checks whether

network daemons violate rules extracted from protocol specifications [50]. EverParse and follow-up work generate verified parsers and serializers from data-format specifications [51], [52]. Recent work has also explored end-to-end verification from high-level protocol models to production code [53], [54]. These efforts demonstrate the value of formal methods for protocol implementation verification, but they often require substantial manual effort to construct and maintain models, specification languages, or proof scripts. When standards or implementations evolve, the corresponding formal artifacts must also be updated. SpecField does not aim to prove the correctness of an entire protocol or implementation. Instead, it identifies potential inconsistencies between natural-language standards and implementation code, and provides traceable evidence for subsequent confirmation.

VIII. Conclusion

This paper studied how to identify field-level conformance deviations in source code implementations from natural-language protocol standards. We presented SpecField, which uses protocol fields as the linking points between standard text and implementation code, extracts structured rules from standards, locates relevant code evidence, and produces auditable conformance judgments. SpecField requires each judgment to be traceable to concrete standard spans, code contexts, and validation results, thereby reducing the risk of misclassification. Across multiple real protocol implementations, we submitted 204 reports, of which 126 were confirmed or fixed by maintainers, and discovered two previously unknown CVEs. These results show that starting from field requirements and combining them with code evidence is an effective way to uncover standard deviations in protocol implementations.

References

[1] T. Dierks and E. Rescorla, "The transport layer security (TLS) protocol version 1.2," RFC 5246, 2008, https://www.rfc-editor.org/rfc/rfc5246.

[2] E. Rescorla, "The transport layer security (TLS) protocol version 1.3," RFC 8446, 2018, https://www.rfc-editor.org/rfc/rfc8446.

[3] OASIS, "MQTT version 3.1.1," 2014, https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html.

[4] ——, "MQTT version 5.0," 2019, https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html.

[5] J. Iyengar and M. Thomson, "QUIC: A UDP-based multiplexed and secure transport," RFC 9000, 2021, https://www.rfc-editor.org/rfc/rfc9000.

[6] M. Thomson and S. Turner, "Using TLS to secure QUIC," RFC 9001, 2021, https://www.rfc-editor.org/rfc/rfc9001.

[7] J. Iyengar and I. Swett, "QUIC loss detection and congestion control," RFC 9002, 2021, https://www.rfc-editor.org/rfc/rfc9002.

[8] S. Bradner, "Key words for use in RFCs to indicate requirement levels," RFC 2119, 1997, https://www.rfc-editor.org/rfc/rfc2119.

[9] B. Leiba, "Ambiguity of uppercase vs lowercase in RFC 2119 key words," RFC 8174, 2017, https://www.rfc-editor.org/rfc/rfc8174.

[10] J. Chen, F. Li, M. Xu, J. Zhou, and W. Huo, "RIBDetector: An RFC-guided inconsistency bug detecting approach for protocol implementations," in *Proceedings of the IEEE International Conference on Software Analysis, Evolution and Reengineering*. IEEE, 2022, pp. 641–651.

[11] M. Zheng, D. Xie, Q. Shi, C. Wang, and X. Zhang, "Validating network protocol parsers with traceable RFC document interpretation," *Proceedings of the ACM on Software Engineering*, vol. 2, no. ISSTA, pp. 1772–1794, 2025.

[12] M. Zheng, C. Wang, X. Liu, J. Guo, S. Feng, and X. Zhang, "RFCAudit: AI agent for auditing protocol implementations against RFC specifications," in *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering*, 2025, pp. 1221–1233.

[13] X. Song, L. Pei, J. Wu, Y. Zeng, G. He, C. Zuo, X. Liu, Q. Zhao, and S. Guo, "ProtocolGuard: Detecting protocol non-compliance bugs via LLM-guided static analysis and dynamic verification," in *Proceedings of the Network and Distributed System Security Symposium*. The Internet Society, 2026.

[14] M. Brunsfeld, "Tree-sitter: An incremental parsing system for programming tools," 2018, https://tree-sitter.github.io/tree-sitter/.

[15] OpenSSL Project, "OpenSSL: Cryptography and SSL/TLS toolkit," 2026, https://www.openssl.org/.

[16] wolfSSL Inc., "wolfSSL embedded SSL/TLS library," 2026, https://www.wolfssl.com/.

[17] Mbed TLS Project, "Mbed TLS," 2026, https://www.trustedfirmware.org/projects/mbed-tls/.

[18] Google, "BoringSSL," 2026, https://boringssl.googlesource.com/boringssl/.

[19] Cloudflare, "quiche: Savoury implementation of the QUIC transport protocol and HTTP/3," 2026, https://github.com/cloudflare/quiche.

[20] T. Tsujikawa, "ngtcp2: An implementation of QUIC," 2026, https://github.com/ngtcp2/ngtcp2.

[21] Eclipse Foundation, "Eclipse Mosquitto: An open source MQTT broker," 2026, https://mosquitto.org/.

[22] wolfSSL Inc., "wolfMQTT client library," 2026, https://github.com/wolfSSL/wolfMQTT.

[23] libcoap Project, "libcoap: A CoAP implementation," 2026, https://libcoap.net/.

[24] X. Sun, F. Dang, S. Jiang, J. Xu, K. Liu, X. Miao, Z. Yang, W. Zhang, H. Lu, Y. Zheng, and Y. Liu, "iPanda: An LLM-based agent for automated conformance testing of communication protocols," 2025, arXiv:2507.00378.

[25] K. Liu, D. Chakraborty, A. Liggesmeyer, and A. Zeller, "AUTOSPEC: Synthesizing precise protocol specs from natural language for effective test generation," 2025, arXiv:2511.17977.

[26] J. Somorovsky, "TLS-Attacker: A framework for testing TLS libraries," in *Proceedings of the 10th USENIX Workshop on Offensive Technologies*, 2016.

[27] M. Maehren, P. Nieting, S. Hebrok, R. Merget, J. Somorovsky, and J. Schwenk, "TLS-Anvil: Adapting combinatorial testing for TLS libraries," in *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, 2022, pp. 215–232.

[28] J. de Ruiter and E. Poll, "Protocol state fuzzing of TLS implementations," in *24th USENIX Security Symposium (USENIX Security 15)*. USENIX Association, 2015, pp. 193–206.

[29] P. Fiterau-Brostean, B. Jonsson, R. Merget, J. de Ruiter, K. Sagonas, and J. Somorovsky, "Analysis of DTLS implementations using protocol state fuzzing," in *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, 2020, pp. 2523–2540.

[30] P. Fiterau-Brostean, B. Jonsson, K. Sagonas, and F. Tåquist, "Automata-based automated detection of state machine bugs in protocol implementations," in *30th Annual Network and Distributed System Security Symposium (NDSS 2023)*. The Internet Society, 2023.

[31] B. Beurdouche, K. Bhargavan, A. Delignat-Lavaud, C. Fournet, M. Kohlweiss, A. Pironti, P.-Y. Strub, and J. K. Zinzindohoue, "A messy state of the union: Taming the composite state machines of TLS," in *Proceedings of the IEEE Symposium on Security and Privacy*, 2015.

[32] H. Asadian, P. Fiterau-Brostean, J. de Ruiter, and E. Poll, "Monitor-based testing of network protocol implementations," in *Proceedings of the IEEE International Conference on Software Testing, Verification and Validation*. IEEE, 2024, pp. 403–414.

[33] M. Piraux and O. Bonaventure, "Observing the evolution of QUIC implementations," in *Proceedings of the Workshop on the Evolution, Performance, and Interoperability of QUIC*, 2018, pp. 8–14.

[34] C. Brubaker, S. Jana, B. Ray, S. Khurshid, and V. Shmatikov, "Using Frankencerts for automated adversarial testing of certificate validation in SSL/TLS implementations," in *Proceedings of the IEEE Symposium on Security and Privacy*, 2014.

[35] Y. T. Chen and Z. Su, "Guided differential testing of certificate validation in SSL/TLS implementations," in *Proceedings of the 10th Joint Meeting on Foundations of Software Engineering*, 2015, pp. 793–804.

[36] C. Chen, C. Tian, Z. Duan, and L. Zhao, "RFC-directed differential testing of certificate validation in SSL/TLS implementations," in *Proceedings of the 40th International Conference on Software Engineering*, 2018, pp. 859–870.

[37] Y. Chen, H. Su, Y. He, S. Jana, S. Chaudhuri, and X. Wang, "HVLearn: Automated black-box analysis of hostname verification in SSL/TLS

implementations," in *Proceedings of the IEEE Symposium on Security and Privacy*, 2017.

[38] S. Y. Chau, O. Chowdhury, E. Hoque, H. Ge, A. Kate, C. Nita-Rotaru, and N. Li, "SymCerts: Practical symbolic execution for exposing noncompliance in X.509 certificate validation implementations," in *Proceedings of the IEEE Symposium on Security and Privacy*, 2017, pp. 503–520.

[39] V.-T. Pham, M. Böhme, and A. Roychoudhury, "AFLNet: A greybox fuzzer for network protocols," in *Proceedings of the IEEE International Conference on Software Testing, Verification and Validation*, 2020, pp. 460–465.

[40] R. Natella, "StateAFL: Greybox fuzzing for stateful network servers," *Empirical Software Engineering*, vol. 27, no. 7, p. 191, 2022.

[41] R. Natella and V.-T. Pham, "ProFuzzBench: A benchmark for stateful protocol fuzzing," in *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2021, pp. 662–665.

[42] J. Ba, M. Böhme, Z. Mirzamomen, and A. Roychoudhury, "Stateful greybox fuzzing," in *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, 2022, pp. 3255–3272.

[43] B. Blanchet, "Modeling and verifying security protocols with the applied pi calculus and ProVerif," *Foundations and Trends in Privacy and Security*, vol. 1, no. 1–2, pp. 1–135, 2016.

[44] ——, "A computationally sound mechanized prover for security protocols," *IEEE Transactions on Dependable and Secure Computing*, vol. 5, no. 4, pp. 193–207, 2008.

[45] C. Cremers, M. Horvat, J. Hoyland, S. Scott, and T. van der Merwe, "A comprehensive symbolic analysis of TLS 1.3," in *Proceedings of the ACM Conference on Computer and Communications Security*, 2017.

[46] J. R. Lewis and B. Martin, "Cryptol: High assurance, retargetable crypto development and validation," Galois Connections white paper, 2003. [Online]. Available: https://cryptol.net/files/cryptol_whitepaper.pdf

[47] R. Dockins, A. Foltzer, J. Hendrix, B. Huffman, D. McNamee, and A. Tomb, "Constructing semantic models of programs with the Software Analysis Workbench," in *Proceedings of the International Conference on Verified Software: Theories, Tools, and Experiments*, 2016.

[48] A. Chudnov, N. Collins, B. Cook, J. Dodds, B. Huffman, C. MacCárthaigh, S. Magill, E. Mertens, E. Mullen, S. Tasiran, A. Tomb, and E. M. Westbrook, "Continuous formal verification of amazon s2n," in *Proceedings of the International Conference on Computer Aided Verification*, 2018, pp. 430–446.

[49] J. Guan, H. Li, X. D. Li, X. L. Wang, B. Wang, Q. Wang, S. Qin, M. He, M. Armanuzzaman, and Z. Zhao, "Formally verifying the state machine of TLS 1.3 handshake in OpenSSL," in *Proceedings of the IEEE Conference on Computer Communications*, 2025.

[50] J. Song, T. Ma, C. Cadar, and P. Pietzuch, "Rule-based verification of network protocol implementations using symbolic execution," in *Proceedings of the International Conference on Computer Communications and Networks*, 2011, pp. 1–8.

[51] T. Ramananandro, A. Delignat-Lavaud, C. Fournet, N. Swamy, T. Chajed, N. Kobeissi, and J. Protzenko, "EverParse: Verified secure Zero-Copy parsers for authenticated message formats," in *28th USENIX Security Symposium (USENIX Security 19)*. USENIX Association, 2019, pp. 1465–1482.

[52] N. Swamy, T. Ramananandro, A. Rastogi, I. Spiridonova, H. Ni, D. Malloy, J. Vazquez, M. Tang, O. Cardona, and A. Gupta, "Hardening attack surfaces with formally proven binary format parsers," in *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2022, pp. 31–45.

[53] J. C. Pereira, T. Klenze, S. Giampietro, M. Limbeck, D. Spiliopoulos, F. A. Wolf, M. Eilers, C. Sprenger, D. Basin, P. Müller, and A. Perrig, "Protocols to code: Formal verification of a next-generation internet router," 2024, arXiv:2405.06074.

[54] L. Arquint, M. Schwerhoff, V. Mehta, and P. Müller, "A generic methodology for the modular verification of security protocol implementations," in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*, 2023, extended version: arXiv:2212.02626.